

**Ejercicio 1**

```
def lee_proyectos(ruta_fichero: str) -> list[Proyecto]:
    res = []
    with open (ruta_fichero, encoding="utf-8") as f:
        lector = csv.reader(f)
        next(lector)
        for id,titulo,categoria,fecha_inicio,fecha_fin,objetivo,recompensas in lector:
            id = id.strip()
            fecha_inicio = parsea_fecha (fecha_inicio)
            fecha_fin = parsea_fecha (fecha_fin)
            objetivo = float(objetivo)
            recompensas = parsea_recompensas(id[:2], recompensas)
            res.append(Proyecto(id, titulo, categoria, fecha_inicio, fecha_fin, \
                                objetivo, recompensas))
        return res

def parsea_fecha(fecha_str: str) -> date:
    return datetime.strptime(fecha_str, "%Y-%m-%d").date()

def parsea_recompensas (origen: str, recompensas_str: str) -> list[Recompensa]:
    res = []
    origen = origen.upper()
    separador = obtener_separador(origen)
    trozos = recompensas_str.split(separador)
    for trozo in trozos:
        if origen == "KS":
            recompensa = parsear_recompensa_KS(trozo)
        elif origen == "CF":
            recompensa = parsear_recompensa_CF(trozo)
        if recompensa:
            res.append(recompensa)
    return res

def obtener_separador (origen: str) -> str:
    separador = ""
    if origen == "KS":
        separador = ";"
    elif origen == "CF":
        separador = "|"
    return separador

def parsear_recompensa_KS(recompesa_str: str) -> Recompensa:
    separador = ":"
    importe, patrocinadores, nombre = recompesa_str.split(separador)
```



```
    return Recompensa(nombre, float(importe), int(patrocinadores))

def parsear_recompensa_CF(recompesa_str: str) -> Recompensa:
    separador = "@"
    nombre, importe, patrocinadores = recompesa_str.split(separador)
    return Recompensa(nombre, float(importe), int(patrocinadores))

## solución alternativa para el parseo de recompensa sin funciones auxiliares
def parsea_recompensas2 (origen: str, recompensas_str: str) -> list[Recompensa]:
    res = []
    # Detectar formato por el prefijo del ID
    if origen.startswith('CF'):
        # Formato CF: nombre@importe@patrocinadores
        bloques = recompensas_str.split('|')
        for bloque in bloques:
            recompensa = parsea_bloque_recompensa(bloque, '@', (0, 1, 2))
            if recompensa:
                res.append(recompensa)
    elif origen.startswith('KS'):
        # Formato KS: importe:patrocinadores:nombre
        bloques = recompensas_str.split(';')
        for bloque in bloques:
            recompensa = parsea_bloque_recompensa(bloque, ':', (2, 0, 1))
            if recompensa:
                res.append(recompensa)
    return res

def parsea_bloque_recompensa(bloque: str, delimitador: str,
                             indices: tuple[int, int, int]) -> Recompensa:
    datos = bloque.split(delimitador)
    idx_nombre, idx_importe, idx_patros = indices
    nombre = datos[idx_nombre]
    importe = float(datos[idx_importe])
    patros = int(datos[idx_patros])
    return Recompensa(nombre, importe, patros)
```

Test

```
def mostrar_iterable(iterable: Iterable) -> None:
    for idx, it in enumerate(iterable, 1):
        print(f"\t{idx}. {it}")

def test_lee_proyectos(proyectos: list[Proyecto]) -> None:
    print("lee_proyectos")
    print(f"Total del proyectos {len(proyectos)}")
```



```
print("Los dos primeros")
mostrar_iterable(proyectos[:2])
print("Los dos últimos")
mostrar_iterable(proyectos[-2:])

def main():
    proyectos = lee_proyectos(r"data\crowdfunding.csv")
    test_lee_proyectos(proyectos)

if __name__ == '__main__':
    main()
```

Ejercicio 2

```
def proyectos_exitosos(proyectos: list[Proyecto], fecha_referencia: date | None = None) \
    -> list[tuple[str, float]]:
    res = []
    for p in proyectos:
        if esta_en_fecha(p, fecha_referencia):
            r = recaudacion(p)
            if r >= p.objetivo:
                res.append((p.titulo, r))
    return res

def recaudacion(proyecto: Proyecto) -> int:
    return sum(r.importe * r.patrocinadores for r in proyecto.recompensas)

def esta_en_fecha(proyecto: Proyecto, fecha_referencia: date | None) -> bool:
    if fecha_referencia == None:
        fecha_referencia = datetime.now().date()
    return proyecto.fecha_fin <= fecha_referencia

## Solución alternativa por comprensión
## Menos eficiente porque repite el cálculo de la recaudación
def proyectos_exitosos(proyectos: list[Proyecto], fecha_referencia: date | None = None) \
    -> list[tuple[str, float]]:
    return [(p.titulo, recaudacion(p)) \
            for p in proyectos \
            if esta_en_fecha(p, fecha_referencia) and recaudacion(p) >= p.objetivo]
```

Test

```
def test_proyectos_exitosos(proyectos: list[Proyecto], fecha_referencia: \
    date | None = None) -> None:
```



```
print("Proyectos exitosos")
res = proyectos_exitosos(proyectos, fecha_referencia)
print(f"Proyectos encontrados: {len(res)} (finalizados antes de {fecha_referencia})")
mostrar_iterable(sorted(res))

def main():
    proyectos = lee_proyectos(r"data\crowdfunding.csv")
    test_proyectos_exitosos(proyectos, date(2024, 12, 31))

if __name__ == '__main__':
    main()
```

Ejercicio 3

```
def top_n_recompensas_mas_populares(proyectos: list[Proyecto], n: int = 3) \
    -> list[Recompensa]:
    gen= ((p.titulo, r.nombre, r.patrocinadores)\
          for p in proyectos for r in p.recompensas)
    return sorted(gen, key=lambda t:t[2], reverse=True)[:n]

## Solución alternativa con bucles
## Menos eficiente en memoria, porque requiere una lista en lugar de generar
def top_n_recompensas_mas_populares(proyectos: list[Proyecto], n: int = 3) \
    -> list[Recompensa]:
    res = []
    for p in proyectos:
        for r in p.recompensas:
            res.append((p.titulo, r.nombre, r.patrocinadores))
    return sorted(res, key=lambda t:t[2], reverse=True)[:n]
```

Test

```
def test_top_n_recompensas_mas_populares(proyectos: list[Proyecto], n: int = 3) \
    -> list[Recompensa]:
    print(f"top_n_recompensas_mas_populares con n={n}")
    res = top_n_recompensas_mas_populares(proyectos, n)
    mostrar_iterable(res)

def main():
    proyectos = lee_proyectos(r"data\crowdfunding.csv")
    test_top_n_recompensas_mas_populares(proyectos)

if __name__ == '__main__':
    main()
```

**Ejercicio 4**

```
def proyecto_con_mayor_numero_patrocinadores(proyectos: list[Proyecto], \
    categoria: str | None = None) -> tuple[str, int]:
    gen = ((p.titulo, patrocinadores(p))\
        for p in proyectos \
            if categoria == None or categoria == p.categoria)
    return max(gen, key=lambda t:t[1])

def patrocinadores(proyecto: Proyecto) -> int:
    return sum(r.patrocinadores for r in proyecto.recompensas)
```

Test

```
def test_proyecto_con_mayor_numero_patrocinadores(proyectos: list[Proyecto], \
    categoria: str | None = None) -> None:
    print("proyecto_con_mayor_numero_patrocinadores")
    print(f"Con filtro de categoría= {categoria}")
    res = proyecto_con_mayor_numero_patrocinadores(proyectos, categoria)
    print(f"\t{res}")

def main():
    proyectos = lee_proyectos(r"data\crowdfunding.csv")
    test_proyecto_con_mayor_numero_patrocinadores(proyectos)
    test_proyecto_con_mayor_numero_patrocinadores(proyectos, "Tecnología")

if __name__ == '__main__':
    main()
```

Ejercicio 5

```
def media_recaudacion_por_categoria(proyectos: list[Proyecto]) -> dict[str, float]:
    d = agrupa_proyectos_por_categoria(proyectos)
    return {categoria: statistics.mean(recaudacion(p) for p in lista_proyectos)\
        for categoria, lista_proyectos in d.items()}

def agrupa_proyectos_por_categoria(proyectos: list[Proyecto]) -> dict[str, list[Proyecto]]:
    d = defaultdict(list)
    for p in proyectos:
        d[p.categoria].append(p)
    return d
```



#Solución con bucles y sin usar statistics.mean

```
def media_recaudacion_por_categoria2(proyectos: list[Proyecto]) -> dict[str, float]:
    d = agrupa_proyectos_por_categoria(proyectos)
    res = dict()
    for categoria, lista_proyectos in d.items():
        res[categoria] = sum(recaudacion(p) for p in lista_proyectos) /
            len(lista_proyectos)
    return res
```

Test

```
def test_media_recaudacion_por_categoria(proyectos: list[Proyecto]):
    print("media_recaudacion_por_categoria")
    res = media_recaudacion_por_categoria(proyectos)
    mostrar_iterable(sorted(res.items()))

def main():
    proyectos = lee_proyectos(r"data\crowdfunding.csv")
    test_media_recaudacion_por_categoria(proyectos)

if __name__ == '__main__':
    main()
```

Ejercicio 6

#Solución 1: diccionario anidado

```
def recaudacion_mensual_por_anyo(proyectos: list[Proyecto]) \
    -> list[tuple[int, list[tuple[int, float]]]]:
    d = agrupa_proyectos_por_anyo(proyectos)
    return sorted((anyo, recaudacion_por_mes(lista_proyectos))
                  for anyo, lista_proyectos in d.items())

def agrupa_proyectos_por_anyo(proyectos: list[Proyecto]) -> dict[int, list[Proyecto]]:
    d = defaultdict(list)
    for p in proyectos:
        d[p.fecha_inicio.year].append(p)
    return d

def recaudacion_por_mes(proyectos: list[Proyecto]) -> list[tuple[int, float]]:
    d = agrupa_proyectos_por_mes(proyectos)
    return sorted((mes, sum(recaudacion(p) for p in lista_proyectos))
                  for mes, lista_proyectos in d.items())
)
```



```
def agrupa_proyectos_por_mes(proyectos: list[Proyecto]) -> dict[int, list[Proyecto]]:
    d = defaultdict(list)
    for p in proyectos:
        d[p.fecha_inicio.month].append(p)
    return d
```

#Solución 2: Usando una clave compuesta.

```
def recaudacion_mensual_por_ano2(proyectos: list[Proyecto]) \
    -> list[tuple[int, list[tuple[int, float]]]]:
    # Diccionario anidado o con clave compuesta (año, mes) -> total
    d = recaudacion_por_ano_mes(proyectos)

    # Reestructurar para la salida
    # 1. Agrupar por años
    por_ano = defaultdict(list)
    for (anyo, mes), total in d.items():
        por_ano[anyo].append((mes, total))

    # 2. Crear lista final ordenada
    return [(anyo, sorted(lista_mes_recaud, key=lambda x: x[0]))\
            for anyo, lista_mes_recaud in por_ano.items()]
    ## Alternativa con bucles
    # res = []
    # for anyo, lista_mes_recaud in sorted(por_ano.items()):
    #     lista_meses=sorted(lista_mes_recaud, key=lambda x: x[0])
    #     res.append ((anyo, lista_meses))
    # return res

def recaudacion_por_ano_mes (proyectos: list[Proyecto]) -> dict[tuple[int, int], float]:
    res = defaultdict(float)
    for p in proyectos:
        anyo = p.fecha_inicio.year
        mes = p.fecha_inicio.month
        res[(anyo, mes)] += recaudacion(p)
    return res
```

Test

```
def test_recaudacion_mensual_por_ano(proyectos: list[Proyecto]) -> None:
    print("recaudacion_mensual_por_ano")
    res = recaudacion_mensual_por_ano(proyectos)
    for anyo, lista in res:
        print(f"{anyo}")
        mostrar_iterable(lista)
```



```
def main():  
    proyectos = lee_proyectos(r"data\crowdfunding.csv")  
    test_recaudacion_mensual_por_anyo(proyectos)  
  
if __name__ == '__main__':  
    main()
```